

PROYECTO INTERMODULAR

Desarrollo de Aplicaciones Multiplataforma (DAM)

ZYRO

Software de Gestión Integral para Hostelería

Autor:

Raúl Oliva Padilla

Jerez de la Frontera, 2026

Índice:

1. Identificación de la Necesidad.
 - 1.1. Análisis del sector y tipos de empresas.
 - 1.1.1. Tipos de establecimientos.
 - 1.1.2. Características comunes del sector.
 - 1.2. Necesidades detectadas / Problema real a solucionar.
 - 1.3. Idea del proyecto y justificación.
 - 1.3.1. Idea.
 - 1.3.2. Justificación.
 - 1.4. Cliente objetivo.
 - 1.5. Guion inicial del trabajo y fases previstas.
2. Diseño Técnico.
 - 2.1. Arquitectura del Software — Diagramas UML.
 - 2.1.1. Diagrama de Casos de Uso.
 - 2.1.2. Diagrama de Clases.
 - 2.1.3. Diagrama de Secuencia — Flujo de Venta.
 - 2.2. Diseño de la base de datos.
 - 2.3. Mockups / Prototipos de interfaz.
 - 2.4. Viabilidad técnica y económica.
 - 2.4.1. Stack Tecnológico Propuesto.
 - 2.4.2. Arquitectura de Alto Nivel.
 - 2.4.3. Riesgos Técnicos y Mitigación.
 - 2.4.4. Viabilidad Económica.
 - 2.5. Criterios de Calidad.

3. Planificación, Presupuesto y Riesgos.

3.1. Cronograma — Diagrama de Gantt.

3.2. Secuencia de tareas y responsables.

Fase 1: Análisis y Definición.

Fase 2: Módulo base y TPV.

Fase 3: Gestión de Stock.

Fase 4: Integración y Rentabilidad.

Fase 5: Reportes y Dashboard.

Fase 6: Pruebas, Ajustes y Lanzamiento.

3.3. Presupuesto del proyecto.

3.3.1. Costes de Personal.

3.3.2. Costes de Infraestructura y Herramientas.

3.3.3. Proyección de Ingresos.

3.4. Gestión de riesgos y medidas preventivas.

4. Cierre y Evaluación del Proyecto.

4.1. Indicadores de calidad y evaluación.

4.1.1. Indicadores Funcionales.

4.1.2. Indicadores Técnicos de Rendimiento.

4.1.3. Indicadores de Experiencia de Usuario (UX).

4.1.4. Indicadores de Evaluación del Proyecto.

4.2. Gestión de cambios e incidencias.

4.2.1. Proceso de Gestión de Cambios.

4.2.2. Registro de Cambios Previsibles.

4.2.3. Proceso de Gestión de Incidencias.

4.3. Conclusiones del Proyecto.

4.3.1. Valoración del Objetivo Alcanzado.

4.3.2. Aprendizajes Técnicos.

4.3.3. Aprendizajes de Gestión de Proyectos.

4.3.4. Limitaciones y Líneas de Trabajo Futuro.

4.3.5. Reflexión Personal.

4.4. Bibliografía y webgrafía.

4.4.1. Bibliografía.

4.4.2. Documentación Técnica Oficial.

4.4.3. Webgrafía y Fuentes Sectoriales.

5. Anexos.

5.1. Anexo 1.

5.2. Anexo 2.

1. IDENTIFICACIÓN DE LA NECESIDAD

1.1. Análisis del Sector y Tipos de Empresas

El sector de la hostelería en España es un pilar fundamental de la economía nacional, caracterizado por su dinamismo, alta competitividad y unos márgenes de beneficio ajustados. Se compone principalmente de los siguientes tipos de establecimientos:

1.1.1. Tipos de establecimientos:

- Restaurantes: Desde alta cocina hasta establecimientos informales. Gestionan menús complejos, cartas de vinos y un alto volumen de materias primas perecederas.
- Bares y Cafeterías: Servicio rápido, alto volumen de transacciones de bajo importe y control crítico de existencias como café, refrescos y tapas.
- Pubs y Discotecas: Negocio centrado en la venta de bebidas, requiriendo un control exhaustivo del stock en barra para prevenir mermas y pérdidas.
- Hoteles y Alojamientos: Además de restaurante y bar, gestionan el stock de habitaciones (minibar, amenidades) y múltiples puntos de venta.
- Food Trucks y negocios de comida rápida: Necesitan soluciones móviles y ágiles para controlar inventario y ventas en ubicaciones cambiantes.

1.1.2. Características comunes del sector:

- Gestión de inventario: Manejo de un gran número de referencias perecederas con fechas de caducidad.
- Alta rotación de personal: Necesidad de un sistema intuitivo que requiera poca formación.
- Estacionalidad: Picos de actividad que exigen una gestión flexible y en tiempo real.
- Fragmentación: Predominio de pymes y microempresas donde el propietario realiza múltiples funciones.
- Digitalización incipiente: Muchos negocios aún dependen de métodos manuales o software obsoleto no integrado.

1.2. Necesidades Detectadas / Problema Real a Solucionar

Tras el análisis sectorial, se identifican varias necesidades no suficientemente cubiertas por las soluciones existentes para la pequeña y mediana empresa hostelera:

1. Falta de Integración y Visión Global: Es común encontrar un TPV para cobrar, un Excel para el stock y notas en papel para los pedidos. Esta desconexión genera trabajo duplicado y errores frecuentes.
2. Gestión Ineficiente del Stock y Desperdicio Alimentario: Desconocimiento del nivel real de existencias, dificultad para calcular costes de platos e imposibilidad de trazar lotes y controlar caducidades.
3. Ineficiencia Operativa: Las tareas manuales de recuento de caja, cálculo de consumos y preparación de pedidos consumen horas valiosas.
4. Toma de Decisiones Basada en la Intuición: La falta de datos claros e informes automáticos impide una gestión estratégica basada en datos.

Problema principal: Los hosteleros carecen de una herramienta única, asequible y fácil de usar que integre la gestión de ventas y stock, les proporcione control financiero en tiempo real y les ayude a reducir pérdidas, optimizando la rentabilidad de su negocio.

1.3. Idea del Proyecto y Justificación

1.3.1. Idea:

Desarrollar un software de gestión integral para hostelería en la nube "Zyro" que combine un TPV robusto con un módulo de control de stock inteligente y reportes de negocio en una única plataforma SaaS. Proponiendo un módulo de inteligencia artificial que analiza el histórico de ventas, la estacionalidad, eventos locales, el clima e incluso las puntuaciones de los clientes para predecir la demanda de cada ingrediente por día, dando así un valor diferencial a "ZYRO".

1.3.2. Justificación:

- Unificación y Sencillez: Al integrar todas las funciones en un solo sistema, se elimina la redundancia de trabajo y se reduce la curva de aprendizaje.
- Enfoque en la Rentabilidad: La asignación de costes a ingredientes y el cálculo automático del margen de cada plato permite decisiones informadas sobre precios.
- Reducción del Desperdicio: El sistema alertará sobre productos próximos a caducar, reduciendo mermas y mejorando la sostenibilidad.

- Accesibilidad para la Pyme: Modelo SaaS con suscripción mensual baja, sin inversión inicial en hardware ni licencias.
- Decisiones basadas en datos: Los dashboards e informes automáticos convertirán los datos del día a día en información accionable.

1.4. Cliente Objetivo

Características	Descripción
Cliente Principal	Pequeñas y medianas empresas del sector hostelero en España
Tipo de Negocio	Bares, restaurantes familiares, cafeterías y food trucks
Tamaño	Negocios con entre 1 y 10 empleados
Nivel Tecnológico	Conocimientos básicos de informática, actualmente usando TPVs básicos o Excel
Necesidad Prioritaria	Simplificar la gestión diaria, controlar costes y existencias, fácil de implementar
Sensibilidad al Precio	Prefieren suscripción mensual baja y sin compromisos a largo plazo

1.5. Guion Inicial del Trabajo y Fases Previstas

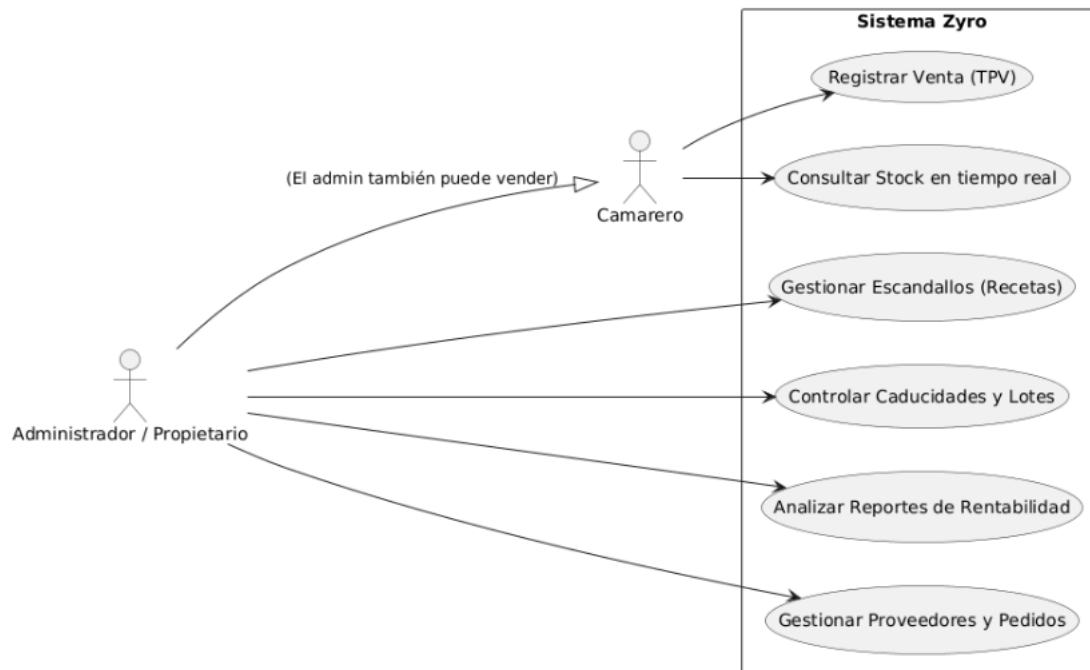
Fase	Nombre	Actividades principales
Fase 1	Análisis y Definición (Sprint 0)	Entrevistas con hosteleros, requisitos funcionales/no funcionales, mockups UI/UX, arquitectura, diseño BD.
Fase 2	Módulo Base y TPV (Sprint 1)	Autenticación y roles, catálogo de productos, TPV básico, registro de ventas y cierre de caja.
Fase 3	Gestión de Stock (Sprint 2)	Alta de proveedores y materias primas, recuentos, ajustes, trazabilidad de lotes y caducidades.
Fase 4	Integración y Rentabilidad (Sprint 3)	Fichas técnicas, descuento automático de stock, cálculo de coste/margen, alertas básicas.
Fase 5	Reportes y Dashboard (Sprint 4)	Dashboard con KPIs, informes predefinidos (ventas, mermas, rentabilidad por producto).
Fase 6	Pruebas y Lanzamiento (Sprint 5)	Testing con usuarios piloto, corrección de bugs, despliegue y lanzamiento MVP v1.0.

2. DISEÑO TÉCNICO

2.1. Arquitectura del Software — Diagramas UML

2.1.1. Diagrama de Casos de Uso

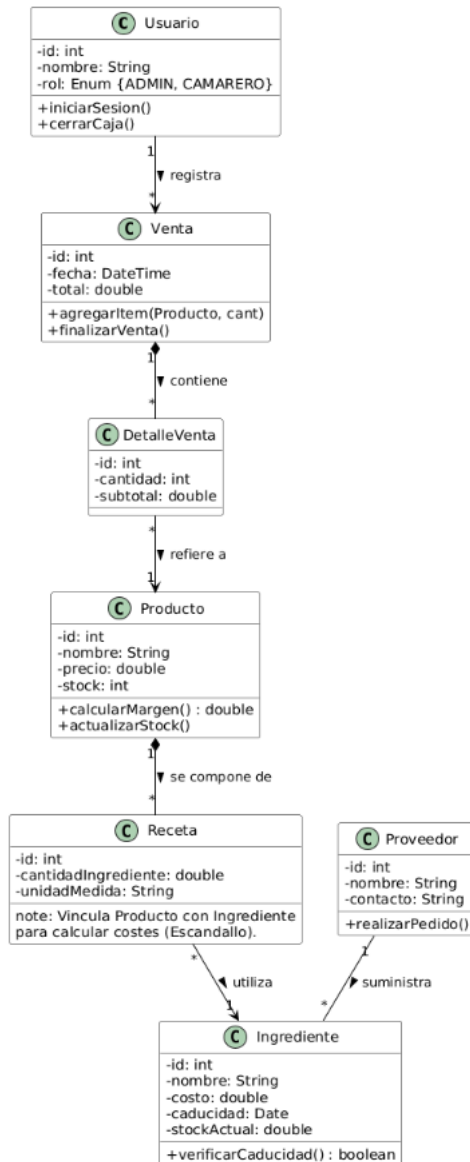
Los actores del sistema son el Administrador/Propietario y el Empleado (Camarero), el administrador tiene acceso a todas las funcionalidades, mientras que el camarero tiene acceso a las operativas de venta e inventario (stock).



2.1.2. Diagrama de Clases

Las clases principales del modelo de dominio son:

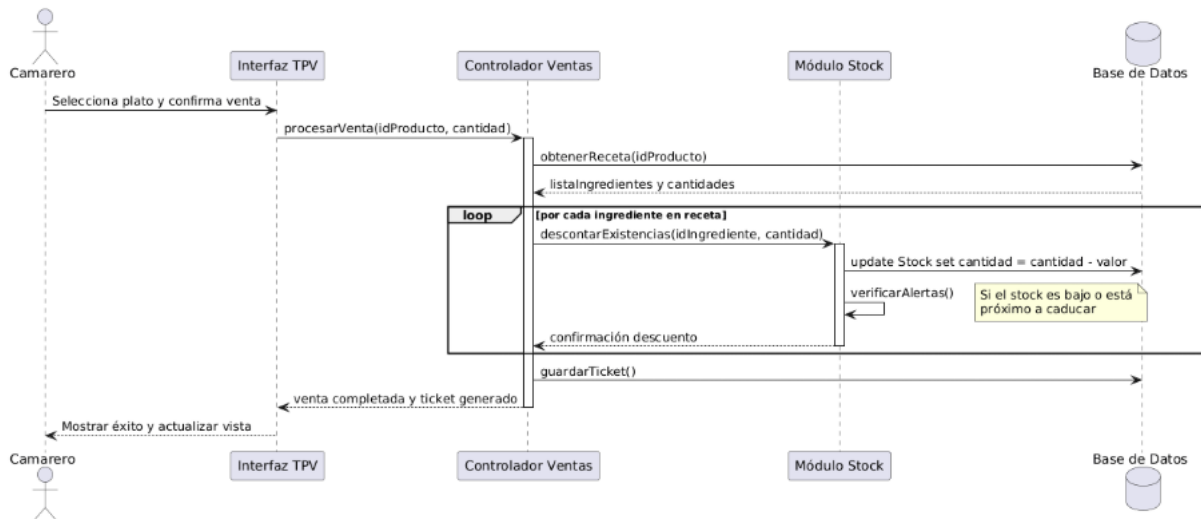
Clase	Atributos principales	Métodos clave
Usuario	id, nombre, rol (ADMIN/CAMARERO)	iniciarSesion(), cerrarCaja()
Producto	id, nombre, precio, stock	calcularMargen(), actualizarStock()
Venta	id, fecha, total	agregarItem(), finalizarVenta()
DetalleVenta	id, cantidad, subtotal	—
Ingrediente	id, nombre, costo, caducidad, stockActual	verificarCaducidad()
Proveedor	id, nombre, contacto	realizarPedido()
Receta	id, cantidadIngrediente, unidadMedida	Vincula Producto con Ingrediente (Escandallo)



2.1.3. Diagrama de Secuencia — Flujo de Venta

El flujo de una venta en Zyro sigue la siguiente secuencia entre los componentes del sistema:

- El camarero selecciona un plato en la Interfaz TPV y confirma la venta.
- El Controlador de Ventas recibe procesarVenta(idProducto, cantidad).
- Se obtiene la receta del producto y la lista de ingredientes necesarios.
- Por cada ingrediente, el Módulo de Stock descuenta las existencias y verifica alertas (stock bajo o próxima caducidad).
- Se guarda el ticket en la Base de Datos y se muestra confirmación al camarero.



2.2. Diseño de la Base de Datos

Se utiliza un modelo relacional normalizado en PostgreSQL hasta 3NF para evitar redundancias. Las características técnicas de la BD son:

1. Claves primarias autoincrementales (SERIAL/BIGSERIAL).
2. Claves foráneas con restricciones ON DELETE/UPDATE para garantizar integridad referencial.
3. Índices en campos de búsqueda frecuente para optimizar el rendimiento.
4. Triggers para mantenimiento automático de datos derivados.

Las tablas principales del sistema son:

Tabla	Descripción	Relaciones
usuarios	Almacena los datos de autenticación y roles de los empleados.	1:N con ventas
productos	Catálogo de platos/bebidas con precio de venta y margen.	N:M con ingredientes (a través de recetas)
ventas	Cabecera de cada transacción de venta.	1:N con detalle_ventas
detalle_ventas	Líneas de cada venta (producto, cantidad, precio unitario).	N:1 con ventas y productos
ingredientes	Materias primas con coste, stock actual y caducidad.	N:M con productos (recetas), N:1 con proveedores
recetas	Tabla puente que vincula productos con ingredientes y cantidades.	N:1 con productos e ingredientes
proveedores	Datos de contacto y condiciones de los proveedores.	1:N con ingredientes
compras	Registro de pedidos de reposición a proveedores.	N:1 con proveedores

movimientos_inventario	Auditoría de todos los movimientos de stock.	N:1 con ingredientes y ventas
alertas	Notificaciones automáticas del sistema (stock bajo, caducidad).	Relacionada con ingredientes

Ver imagen en Anexo 1.

2.3. Mockups / Prototipos de Interfaz

Se han diseñado cuatro pantallas clave que representan los módulos principales de “ZYRO”:

Pantalla	Descripción funcional
Dashboard Principal	Vista de KPIs (ingresos del día, pedidos totales, ticket medio, clientes atendidos), gráfico de análisis de ventas por hora/categoría/semana y panel de alertas y notificaciones en tiempo real.
TPV (Punto de Venta)	Interfaz táctil dividida en categorías (bebidas, cafetería, entrantes, principales, postres) con búsqueda de productos y carrito de compra con cálculo automático de subtotal, IVA y total.
Gestión de Inventario	Resumen de valor de inventario, productos con stock bajo, alertas de caducidad y total de referencias. Tabla de inventario actual con estado del stock (crítico/correcto/bajo) y fecha de caducidad.
Informes y Estadísticas	Filtros por fecha, empleado y categoría para generar informes. Métricas de ventas totales, ticket medio, transacciones y empleado destacado, con gráficos de evolución y ventas por categoría.

Ver imágenes en Anexo 2.

2.4. Viabilidad Técnica y Económica

2.4.1. Stack Tecnológico Propuesto

Capa	Tecnología	Justificación
Frontend	React.js + Tailwind CSS	Componentización, alto rendimiento y curva de aprendizaje moderada.
Backend	Node.js + Express.js	Ecosistema maduro, asincronía natural para TPV, enorme comunidad.
Base de Datos	PostgreSQL	Fiabilidad transaccional, soporte JSONB, rendimiento en consultas complejas.
Infraestructura	Docker + AWS/DigitalOcean	Portabilidad, escalabilidad horizontal, coste controlable.
Autenticación	JWT + Passport.js	Estándar, fácil implementación y sin estado (stateless).
Control de versiones	Git + GitHub	Imprescindible para colaboración y despliegue continuo.

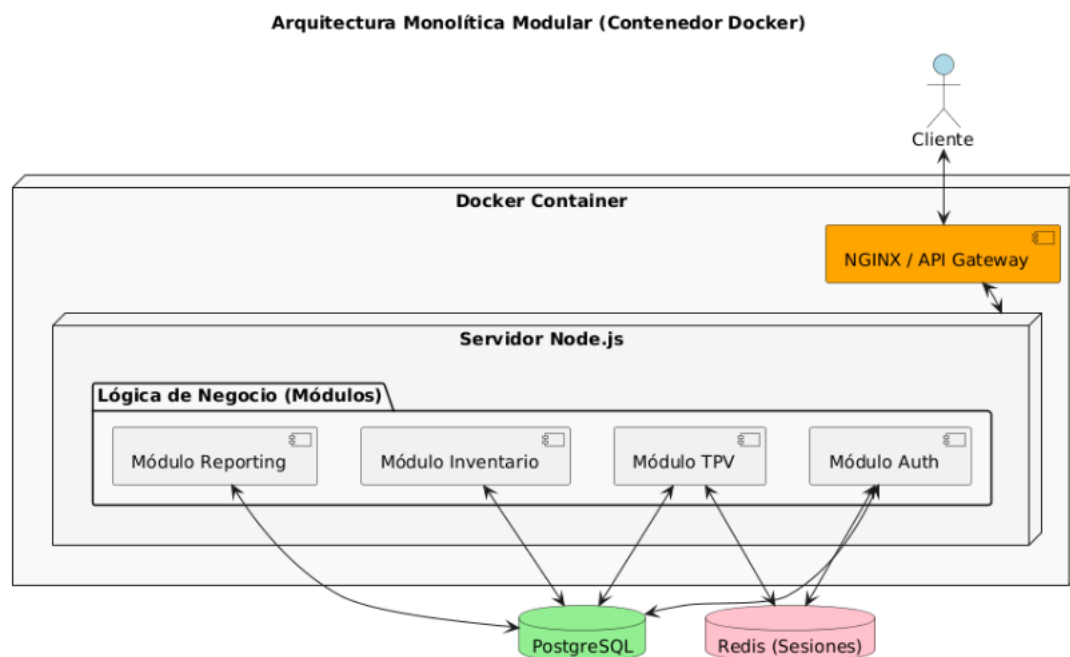
Testing	Jest, Cypress, Supertest	Pruebas unitarias, integración y E2E.
---------	--------------------------	---------------------------------------

2.4.2. Arquitectura de Alto Nivel

Se propone una arquitectura monolítica modular desplegada en contenedores Docker, con posibilidad de migrar a microservicios en el futuro. Los módulos internos son:

- Módulo Auth: Gestión de autenticación, sesiones y control de roles.
- Módulo TPV: Procesamiento de ventas, gestión de comandas y cierre de caja.
- Módulo Inventario: Control de stock, trazabilidad de lotes y alertas.
- Módulo Reporting: Generación de informes, dashboards y KPIs.

El acceso del cliente se realiza a través de NGINX como gateway, que enruta las peticiones al servidor Node.js. La persistencia se gestiona mediante PostgreSQL y las sesiones con Redis.



2.4.3. Riesgos Técnicos y Mitigación

Riesgo	Probabilidad	Impacto	Mitigación
Complejidad en descuento automático de stock	Alta	Alto	Prototipar el algoritmo desde la semana 12 y validar con hostelero real.
Rendimiento en picos de venta	Media	Medio	Uso de Redis para caché y pruebas de carga con K6.

Falta de experiencia en DevOps	Media	Medio	Utilizar PaaS (Railway, Render) para evitar gestión manual de servidores.
Pérdida de datos	Baja	Alto	Copias de seguridad automáticas diarias en la nube.

2.4.4. Viabilidad Económica

El proyecto se desarrolla como TFG unipersonal, por lo que la inversión económica directa es mínima. El modelo de comercialización propuesto es SaaS con tres planes:

Plan	Precio/mes	Funcionalidades clave	Público objetivo
Básico	19 €	TPV, catálogo, informe de ventas simples	Food trucks, bares pequeños
Estándar	29 €	Básico + stock básico, alertas de stock bajo	Cafeterías, restaurantes familiares
Premium	39 €	Estándar + fichas técnicas, rentabilidad, multi-usuario	Restaurantes con carta elaborada

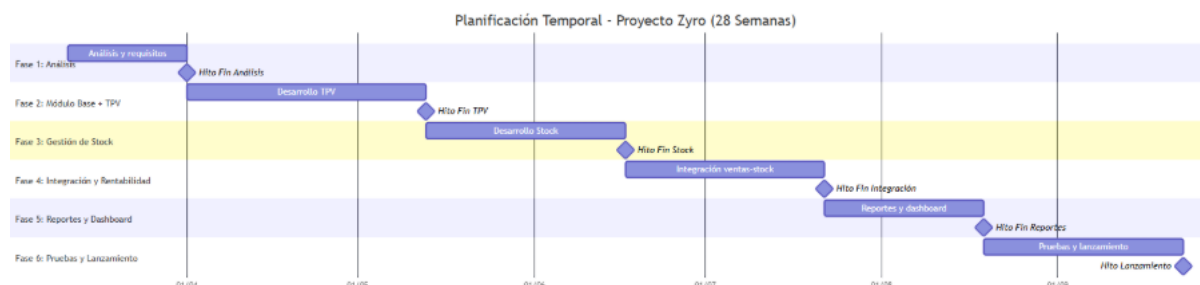
2.5. Criterios de Calidad

Categoría	Criterio	Estándar aceptable
Funcionalidad	Cumplimiento de requisitos	> 95% de funcionalidades implementadas
Funcionalidad	Exactitud de cálculos (costes/ventas)	Tasa de error < 0,5%
Usabilidad	Facilidad de uso sin formación previa	> 80% tareas completadas en primer uso
Usabilidad	Tiempo para completar venta tipo	< 60 segundos
Rendimiento	Tiempo de respuesta API	< 300 ms
Rendimiento	Tiempo de carga de interfaz	< 2 segundos
Rendimiento	Concurrencia	> 10 usuarios simultáneos
Seguridad	Autenticación	Sin vulnerabilidades conocidas
Seguridad	Protección de credenciales	Contraseñas hasheadas con bcrypt (obligatorio)
Fiabilidad	Integridad referencial	Consistencia BD al 100% después de cada venta
Mantenimiento	Documentación técnica	Completa y clara
Disponibilidad	Dispositivos	Diseño responsive

3. PLANIFICACIÓN, PRESUPUESTO Y RIESGOS

3.1. Cronograma — Diagrama de Gantt

La planificación temporal se ha elaborado siguiendo las 6 fases definidas en el apartado 1.5. Está ajustada a un equipo unipersonal con dedicación completa (35-40 h/semana). La duración total estimada es de 28 semanas (≈ 7 meses).



Fases	Semanas	Hito de cierre
Fase 1: Análisis y requisitos	1 – 3	Hito Fin Análisis
Fase 2: Módulo Base + TPV	4 – 9	Hito Fin TPV
Fase 3: Gestión de Stock	10 – 14	Hito Fin Stock
Fase 4: Integración y Rentabilidad	15 – 19	Hito Fin Integración
Fase 5: Reportes y Dashboard	20 – 23	Hito Fin Reportes
Fase 6: Pruebas y Lanzamiento	24 – 28	Hito Lanzamiento

3.2. Secuencia de Tareas y Responsables

El responsable de todas las tareas es Raúl Oliva Padilla. A continuación, se detalla la secuencia de actividades con sus dependencias por fase.

Fase 1: Análisis y Definición (Semanas 1-3, ~245h)

Tarea	Descripción	Duración	Dependencias
1.1	Realizar entrevistas con 3-5 hosteleros para validar necesidades	1 semana	—
1.2	Redactar documento de requisitos funcionales y no funcionales	1 semana	1.1
1.3	Diseñar mockups de las pantallas clave	2 semanas	1.2
1.4	Definir arquitectura software y stack tecnológico definitivo	1 semana	1.2
1.5	Diseñar modelo Entidad-Relación de la base de datos	1 semana	1.2
1.6	Elaborar plan de desarrollo (sprints)	1 semana	1.4, 1.5

Fase 2: Desarrollo del Módulo Base y TPV (Semanas 4-9, ~350h)

Tarea	Descripción	Duración	Dependencias
2.1	Configurar repositorio Git, entorno de desarrollo (Node.js, React, PostgreSQL)	1 semana	1.4, 1.5
2.2	Implementar autenticación JWT y gestión de roles	2 semanas	2.1
2.3	Desarrollar CRUD de categorías y productos	2 semanas	2.2
2.4	Crear interfaz TPV básica (selección de productos, carrito)	2 semanas	2.3
2.5	Implementar registro de ventas y cierre de caja	2 semanas	2.4
2.6	Pruebas unitarias y de integración del módulo TPV	1 semana	2.5

Fase 3: Desarrollo del Módulo de Gestión de Stock (Semanas 10-14, ~245h)

Tarea	Descripción	Duración	Dependencias
3.1	CRUD de proveedores	1 semana	2.1
3.2	CRUD de ingredientes	1.5 semanas	3.1
3.3	Gestión de lotes (cantidad, fechas, estado)	1.5 semanas	3.2
3.4	Implementar recuentos de stock y ajustes manuales	1 semana	3.3
3.5	Interfaz de consulta de stock y movimientos	1 semana	3.4
3.6	Pruebas unitarias e integración del módulo stock	1 semana	3.5

Fase 4: Integración y Cálculo de Rentabilidad (Semanas 15-19, ~245h)

Tarea	Descripción	Duración	Dependencias
4.1	Crear fichas técnicas (recetas) vinculando productos e ingredientes	2 semanas	2.3, 3.2
4.2	Calcular automáticamente coste y margen por plato	1 semana	4.1
4.3	Descontar automáticamente ingredientes del stock al realizar venta	2 semanas	2.5, 4.1
4.4	Implementar alertas básicas (stock bajo, caducidad próxima)	1 semana	4.3
4.5	Pruebas de integración ventas-stock	1 semana	4.3

Fase 5: Desarrollo de Reportes y Dashboard (Semanas 20-23, ~185h)

Tarea	Descripción	Duración	Dependencias
5.1	Diseñar dashboard con KPIs principales (ventas día, margen, top productos)	1.5 semanas	2.5, 4.2

5.2	Desarrollar informes predefinidos (ventas por período, mermas, rentabilidad)	2 semanas	4.2, 4.3
5.3	Añadir gráficos interactivos (Chart.js o similar)	1 semana	5.2
5.4	Pruebas de usabilidad del dashboard	5 días	5.3

Fase 6: Pruebas, Ajustes y Lanzamiento (Semanas 24-28, ~245h)

Tarea	Descripción	Duración	Dependencias
6.1	Ejecutar pruebas unitarias, integración y E2E completas	1.5 semanas	Todo lo anterior
6.2	Realizar pruebas de usabilidad con usuarios piloto (3-5 establecimientos)	1.5 semanas	6.1
6.3	Corregir errores críticos y priorizar mejoras	1 semana	6.2
6.4	Desplegar en producción (Railway/Render) y configurar dominio	1 semana	6.3
6.5	Redactar documentación técnica y manual de usuario	1 semana	6.4
6.6	Preparar presentación y defensa del TFG	1 semana	6.5

3.3. Presupuesto del Proyecto**3.3.1. Costes de Personal**

Al ser un proyecto intermodular desarrollado íntegramente por Raúl Oliva Padilla, no me imputo salario. A efectos de coste de oportunidad, se estima una valoración teórica de 15 €/hora × 980 horas = 14.700 €.

3.3.2. Costes de Infraestructura y Herramientas (Año 1)

Concepto	Proveedor	Coste	Observaciones
Hosting desarrollo	Railway / Render	0 €	Plan gratuito
Hosting producción (post-MVP)	Railway / Render	240 €	Estimado 20 €/mes desde el año 2
Dominio .com	Namecheap / GoDaddy	12 €	Primer año
Certificado SSL	Let's Encrypt	0 €	Automático
Herramientas de diseño	Figma	0 €	Plan gratuito
Gestión de proyectos	Trello / Notion	0 €	Plan gratuito
Control de versiones	GitHub	0 €	Repositorio privado gratuito
Total costes directos (año 1)		12 €	

3.3.3. Proyección de Ingresos (Escenario Post-TFG)

Concepto	Año 1	Año 2	Año 3
Clientes (media)	25	35	50
Ingreso medio mensual	22 €	25 €	28 €
Ingresos recurrentes (anual)	6.600 €	10.500 €	16.800 €
Costes operativos	1.470 €	2.190 €	2.790 €
Resultado neto estimado	5.130 €	8.310 €	14.010 €

* Costes operativos: hosting, dominio, SSL, mantenimiento estimado (10 h/mes a 15 €/h).

3.4. Gestión de Riesgos y Medidas Preventivas

ID	Riesgo	Probabilidad	Impacto
R01	Complejidad técnica en la integración ventas-stock	Alta	Alto
R02	Desviación de plazos por subestimación	Alta	Alto
R03	Problemas de rendimiento en picos de uso	Media	Medio
R04	Falta de feedback de usuarios reales	Media	Alto
R05	Problemas de seguridad (inyección, XSS)	Baja	Alto
R06	Pérdida de datos	Baja	Alto
R07	Falta de motivación / agotamiento del desarrollador	Media	Medio

ID	Medidas preventivas	Medidas de contingencia
R01	- Prototipar el algoritmo de descuento desde la semana 12. - Validar con datos reales de un hostelero.	- Simplificar el modelo inicial (descuento global sin lotes). - Dedicar tiempo extra en la fase 4.
R02	- Planificación con sprints de 2 semanas y revisión semanal. - Priorización estricta (MoSCoW).	- Reducir alcance de funcionalidades no críticas. - Añadir 2 semanas de colchón al final.
R03	- Usar índices en BD desde el inicio. - Pruebas de carga con K6 antes del lanzamiento.	- Implementar caché con Redis. - Optimizar consultas lentas.
R04	- Contactar con 3-5 establecimientos desde la Fase 1. - Ofrecer prueba gratuita a cambio de feedback.	- Realizar pruebas simuladas con datos ficticios. - Usar foros y comunidades de hostelería.
R05	- Validar y sanitizar todas las entradas.	- Realizar escaneo con OWASP ZAP previo al lanzamiento.

	- Usar JWT con expiración y HttpOnly. - Ejecutar npm audit regularmente.	- Corregir vulnerabilidades críticas inmediatamente.
R06	- Copias de seguridad automáticas diarias (proveedor cloud). - Uso de migraciones con control de versiones.	- Restaurar desde backup. - Mantener un registro de cambios.
R07	- Establecer rutina de trabajo con descansos. - Celebrar hitos semanales.	- Reducir temporalmente la dedicación si es necesario. - Buscar apoyo de tutor o compañeros.

4. CIERRE Y EVALUACIÓN DEL PROYECTO

4.1. Indicadores de Calidad y Evaluación

Con el objetivo de garantizar que el producto final cumple los estándares de calidad definidos en la Entrega 2, se han establecido indicadores cuantitativos y cualitativos agrupados en cuatro dimensiones de evaluación: funcional, técnica, de experiencia de usuario y de negocio.

4.1.1. Indicadores Funcionales

Indicador	Descripción	Valor objetivo	Herramienta de medición
Cobertura de requisitos (FR)	Porcentaje de requisitos funcionales implementados y verificados.	≥ 95 %	Checklist de requisitos, revisión manual
Tasa de defectos post-pruebas	Número de bugs críticos detectados en la fase de pruebas E2E.	0 bugs críticos	Cypress + registro de issues en GitHub
Exactitud de cálculo de costes	Desviación entre el coste calculado por Zyro y el coste real de producción.	Error < 0,5 %	Pruebas unitarias Jest sobre motor de cálculo
Exactitud de descuento de stock	Concordancia entre unidades descontadas del stock y ventas reales registradas.	100 % en pruebas	Pruebas de integración con BD de test
Cobertura de código (unit tests)	Porcentaje de líneas de código cubiertas por pruebas unitarias automatizadas.	≥ 70 %	Jest + Istanbul (nyc)

4.1.2. Indicadores Técnicos de Rendimiento

Indicador	Descripción	Valor objetivo	Herramienta
Tiempo de respuesta API (p95)	Latencia del percentil 95 de las peticiones a la API REST bajo carga normal.	< 300 ms	K6 / Postman

Tiempo de carga inicial (FCP)	First Contentful Paint de la aplicación React en red estándar.	< 2 s	Lighthouse / DevTools
Concurrencia soportada	Número de usuarios Simultáneos sin degradación de rendimiento apreciable.	≥ 10 usuarios	K6 con escenario de carga
Disponibilidad del servicio	Porcentaje de tiempo en que el sistema está operativo en producción.	≥ 99 % mensual	UptimeRobot
Tamaño del bundle (JS)	Tamaño del archivo JavaScript principal del frontend tras compilación.	< 500 KB gzip	Webpack Bundle Analyzer

4.1.3. Indicadores de Experiencia de Usuario (UX)

Indicador	Descripción	Valor objetivo	Método
Tasa de éxito en tareas clave (SUS)	Porcentaje de usuarios piloto capaces de completar las 5 tareas clave sin ayuda en la primera sesión.	≥ 80 %	Pruebas de usabilidad presenciales (Fase 6)
Tiempo medio para completar venta	Tiempo transcurrido desde que el camarero selecciona el primer producto hasta que se imprime el ticket.	< 60 segundos	Observación cronometrada en pruebas piloto
Puntuación SUS (System Usability Scale)	Escala estandarizada de usabilidad (0-100) obtenida de los usuarios piloto.	≥ 68 puntos ("bueno")	Cuestionario SUS estándar
Satisfacción general (NPS)	Net Promoter Score de los establecimientos piloto tras 1 semana de uso.	≥ 30	Encuesta de una pregunta post-prueba

4.1.4. Indicadores de Evaluación del Proyecto

Indicador	Descripción	Valor objetivo
Desviación de plazos	Diferencia entre duración planificada (28 semanas) y real.	≤ ±2 semanas
Desviación de esfuerzo	Diferencia entre horas estimadas (980 h) y horas reales invertidas.	≤ ±15 %
Documentación técnica	Compleitud de la documentación.	Completa y revisada por el tutor
Superación de defensa TFG	Calificación obtenida en la presentación ante el tribunal evaluador.	≥ Notable (7/10)

4.2. Gestión de Cambios e Incidencias

Durante el ciclo de vida de un proyecto software, es inevitable que surjan cambios respecto a la planificación inicial y que se registren incidencias. A continuación, se describe el proceso formal adoptado en el proyecto Zyro para gestionar ambos de manera controlada.

4.2.1. Proceso de Gestión de Cambios

Se ha adoptado un proceso de control de cambios ligero pero estructurado, adaptado a la naturaleza unipersonal del proyecto y basado en la metodología ágil:

- **Identificación:** Cualquier cambio de alcance, requisito o diseño técnico se registra como una issue en GitHub con la etiqueta "Change-request".
- **Evaluación de impacto:** Se analiza el impacto en el cronograma (semanas), el esfuerzo adicional (horas) y la prioridad MoSCoW (Must/Should/Could/Won't).
- **Decisión y aprobación:** El cambio es aprobado o rechazado en la reunión de revisión de sprint semanal con el tutor académico.
- **Implementación:** Los cambios aprobados se incorporan al backlog del sprint en curso o del siguiente, actualizando el diagrama de Gantt si afectan al camino crítico.
- **Trazabilidad:** Cada cambio queda vinculado a su commit de Git correspondiente mediante el número de issue (#id), garantizando la trazabilidad completa.

4.2.2. Registro de Cambios Previsibles

Aunque el proyecto está en fase de desarrollo, se anticipan los siguientes cambios respecto a la planificación inicial basándose en los riesgos identificados:

ID Cambio	Origen	Descripción	Impacto estimado
CC-01	Riesgo R02 (plazos)	Posposición de la funcionalidad de módulo multi-usuario (empresas con varios locales).	0 h en TFG
CC-02	Feedback tutor	Inclusión de exportación de informes a PDF/Excel como funcionalidad adicional si el tiempo lo permite.	+10 h Sprint 4

4.2.3. Proceso de Gestión de Incidencias

Las incidencias (bugs, fallos de integración, problemas de rendimiento) se gestionan mediante el siguiente flujo:

Severidad	Definición	Tiempo máx. resolución	Acción
Crítica (P1)	El sistema no puede procesar ventas o pierde datos.	< 4 horas	Parar el sprint y resolver de inmediato. Notificar al tutor.
Alta (P2)	Una funcionalidad principal no funciona correctamente, pero	< 1 día	Incorporar al sprint actual como tarea prioritaria.

	existe workaround.		
Media (P3)	Funcionalidad secundaria incorrecta o degradación de rendimiento no crítico.	< 1 semana	Añadir al backlog del siguiente sprint.
Baja (P4)	Errores de interfaz, textos incorrectos, mejoras menores.	Post-MVP	Acumular en el backlog de mejoras.

Todas las incidencias se documentan en GitHub Issues con: descripción detallada del fallo, pasos para reproducirlo, entorno de prueba, severity/priority y asignación. El cierre de cada issue requiere la evidencia de la prueba que confirma la corrección.

4.3. Conclusiones del Proyecto

4.3.1. Valoración del Objetivo Alcanzado

El proyecto Zyro nació de la detección de una necesidad real y no resuelta en el sector hostelero español: la inexistencia de una solución integral, asequible y fácil de usar para la gestión de ventas, inventario y rentabilidad en la pequeña empresa hostelera.

A lo largo de los apartados anteriores se ha construido una base sólida que abarca desde el análisis del problema y la identificación del cliente objetivo, hasta el diseño técnico completo con diagramas UML, modelo de datos, mockups y análisis de viabilidad, culminando con una planificación detallada, presupuesto y gestión de riesgos. Este último punto 4 cierra el ciclo con los mecanismos de garantía de calidad, el proceso de gestión del cambio y la reflexión final sobre el proyecto.

4.3.2. Aprendizajes Técnicos

- Arquitectura de software: La definición de una arquitectura monolítica modular con vistas a la evolución futura a microservicios ha puesto en práctica conocimientos de diseño de sistemas escalables y mantenibles.
- Stack tecnológico moderno: El trabajo con React.js, Node.js/Express, PostgreSQL y Docker ha supuesto un aprendizaje profundo de las tecnologías más demandadas en el mercado laboral actual.
- Diseño de bases de datos: El modelado relacional normalizado hasta 3NF con integridad referencial, triggers e índices optimizados ha consolidado conocimientos de bases de datos en un contexto real.

- Metodología ágil: La planificación por sprints, la priorización MoSCoW y la gestión iterativa del backlog han demostrado su valor en un proyecto con un único desarrollador.
- DevOps básico: La configuración de entornos Docker y el despliegue en plataformas PaaS como Railway ha introducido conceptos de integración y despliegue continuo.

4.3.3. Aprendizajes en Gestión de Proyectos

- La importancia del análisis previo: Las entrevistas con hosteleros reales en la Fase 1 han sido fundamentales para evitar construir funcionalidades que el cliente no necesita y para validar las hipótesis del proyecto.
- La subestimación del tiempo es un riesgo universal: Las 980 horas estimadas representan el esfuerzo mínimo; las tareas de integración, testing y documentación siempre superan las estimaciones iniciales.
- El valor de los prototipos: Los mockups permitieron detectar inconsistencias en el flujo de usuario antes de escribir una sola línea de código, ahorrando tiempo de desarrollo.
- La gestión del riesgo como hábito: Identificar y planificar respuestas a los riesgos (tablas punto 3.4) ha permitido actuar de forma proactiva ante desviaciones previsibles.

4.3.4. Limitaciones y Líneas de Trabajo Futuro

El MVP de Zyro, aunque completo para las necesidades del cliente objetivo definido, presenta limitaciones que abren líneas de desarrollo futuro:

Limitación actual	Línea de trabajo futuro
Gestión de un único local por cuenta	Módulo multi local/franquicias para cadenas de restaurantes.
Sin integración con TPV hardware físico	Integración con dispositivos de cobro (datáfonos) y lectores de código de barras.
Informes no exportables	Exportación a PDF y Excel de todos los informes analíticos.
Sin módulo de reservas	Integración con gestión de reservas de mesas y agenda de eventos.
Sin app móvil nativa	Aplicación iOS/Android para gestión en movilidad (especialmente para food trucks).
Sin IA/ML predictivo	Módulo de predicción de demanda y sugerencias de pedido a proveedores basado en histórico.

4.3.5. Reflexión Personal

El desarrollo de Zyro como Proyecto Intermodular ha representado mucho más que la aplicación de conocimientos técnicos adquiridos durante el grado. Ha supuesto enfrentarme al ciclo completo de creación de un producto software real: desde la empatía con el usuario y la definición del problema, pasando por las decisiones de arquitectura y diseño, hasta la planificación del tiempo y los recursos con criterios profesionales.

La combinación de un dominio de negocio concreto (la hostelería) con los desafíos técnicos de un sistema integral (integración TPV-stock, cálculo de rentabilidad en tiempo real, trazabilidad de ingredientes) ha constituido un reto que, sin duda, ha acelerado mi madurez como desarrollador de software. Zyro es, en definitiva, la demostración práctica de que el aprendizaje más profundo ocurre cuando se construye algo que responde a un problema real.

4.4. Bibliografía y Webgrafía

4.4.1. Bibliografía

- Pressman, R. S. & Maxim, B. R. (2015). Ingeniería del Software: Un Enfoque Práctico (8ª ed.). McGraw-Hill Education.
- Sommerville, I. (2016). Software Engineering (10th ed.). Pearson.
- Elmasri, R. & Navathe, S. B. (2015). Fundamentos de Sistemas de Bases de Datos (7ª ed.). Pearson Educación.
- Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley.
- Knapp, J., Zeratsky, J. & Kowitz, B. (2016). Sprint: How to Solve Big Problems and Test New Ideas in Just Five Days. Simon & Schuster.
- Norman, D. (2013). The Design of Everyday Things (Revised and Expanded Edition). Basic Books.

4.4.2. Documentación Técnica Oficial

- React Documentation (2024). React – A JavaScript library for building user interfaces. <https://react.dev>
- Node.js Documentation (2024). Node.js v20 LTS. <https://nodejs.org/en/docs>
- Express.js Guide (2024). Express – Fast, unopinionated, minimalist web framework for Node.js. <https://expressjs.com>
- PostgreSQL Documentation (2024). PostgreSQL 16 Documentation. <https://www.postgresql.org/docs/16/>

- Docker Documentation (2024). Docker Docs – Build, share, and run applications. <https://docs.docker.com>
- Jest Documentation (2024). Jest – Delightful JavaScript Testing. <https://jestjs.io/docs/getting-started>
- Cypress Documentation (2024). Cypress – E2E Testing Framework. <https://docs.cypress.io>
- JSON Web Tokens (JWT) Introduction (2024). <https://jwt.io/introduction>

4.4.3. Webgrafía y Fuentes Sectoriales

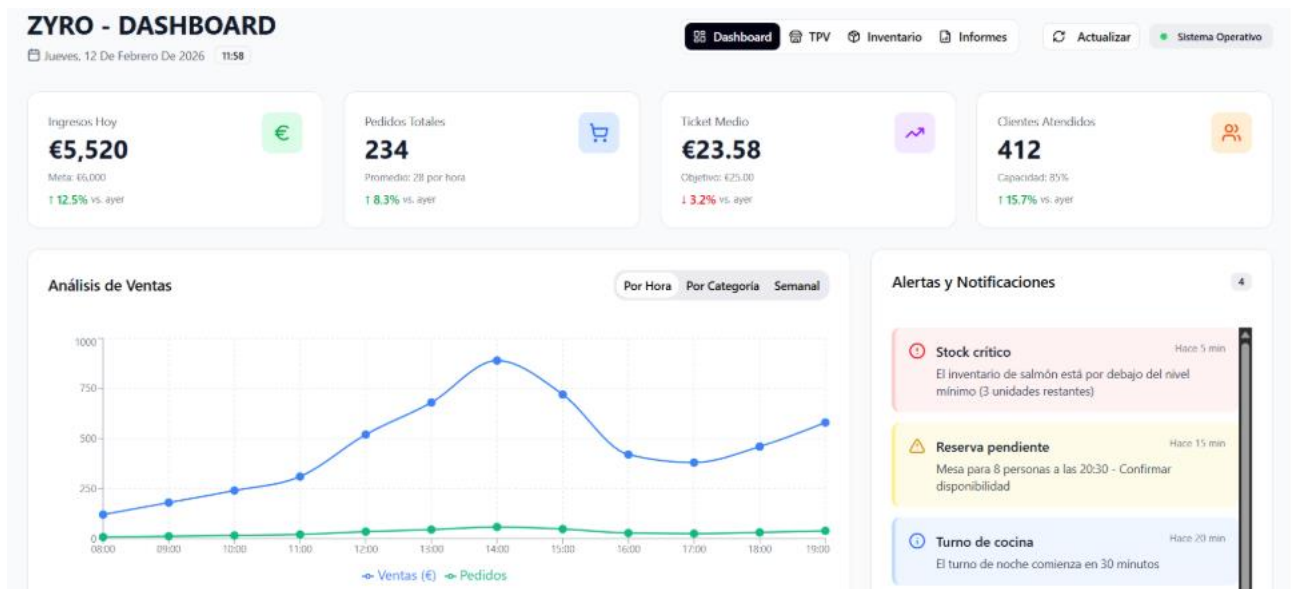
- FEHR – Federación Española de Hostelería (2023). Informe Económico del Sector Hostelero 2023. <https://www.fehr.es>
- Hostelería de España (2024). Radiografía de la hostelería española. <https://www.hosteleria.com>
- INE – Instituto Nacional de Estadística (2024). Directorio Central de Empresas (DIRCE) – Sector Servicios de Comidas y Bebidas. <https://www.ine.es>
- OWASP Foundation (2024). OWASP Top Ten – Web Application Security Risks. <https://owasp.org/www-project-top-ten/>
- Railway Documentation (2024). Railway – Infrastructure, Instantly. <https://docs.railway.app>
- Figma (2024). Figma Design – Collaborative Interface Design Tool. <https://www.figma.com>
- K6 Documentation (2024). k6 – Modern Load Testing Tool. <https://k6.io/docs/>
- Tailwind CSS Documentation (2024). Tailwind CSS – A utility-first CSS framework. <https://tailwindcss.com/docs>

5.2. Anexo 2:

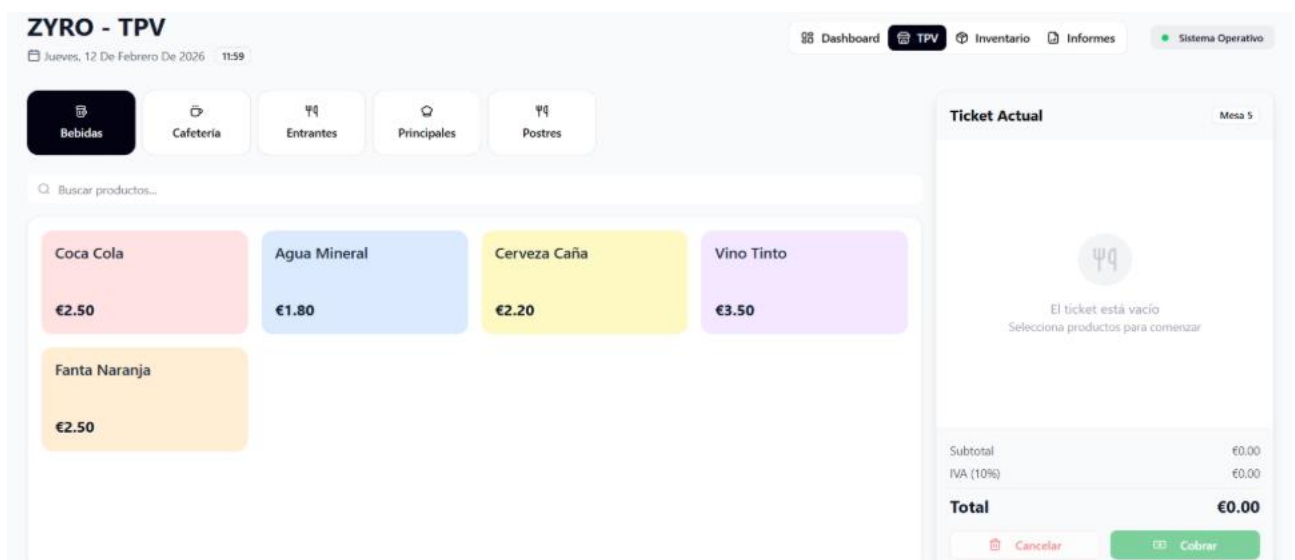
Mockups / Prototipos de Interfaz:

Se navegará entre pantallas clicando en el nombre de la pantalla a elegir en la esquina superior derecha.

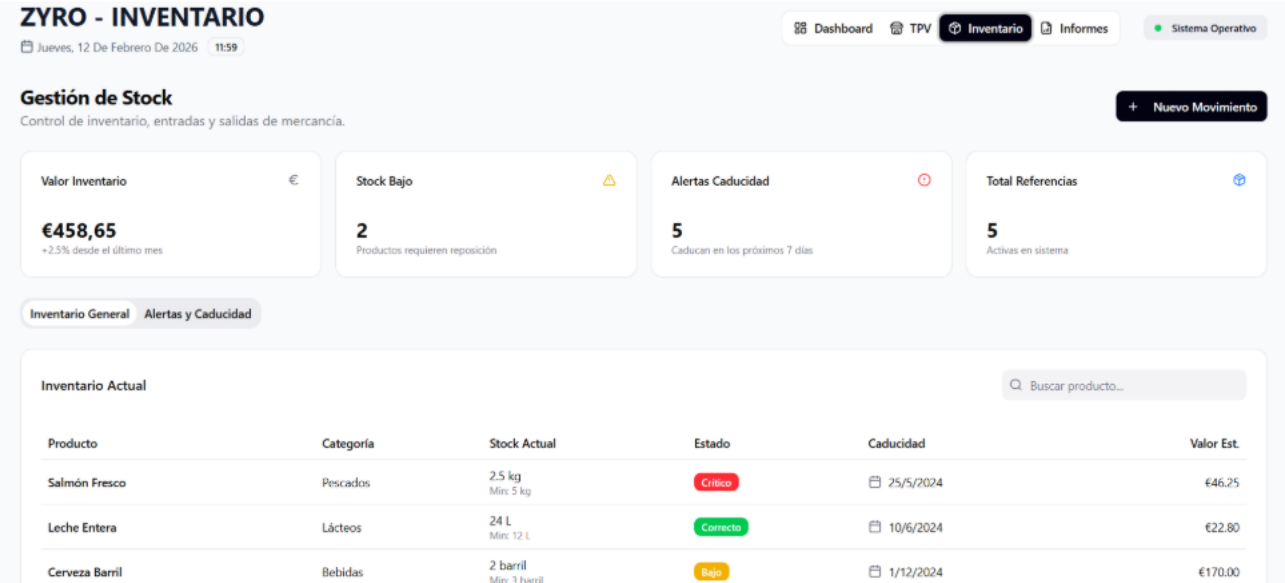
Dashboard principal:



TPV:



Inventario:



Informes:

